

Trabalho Prático – Sistema de Gerenciamento de Hotel com Banco de Dados Relacional

Grupos de até 4 integrantes (obrigatoriamente os mesmos integrantes das duplas da prova)

Entrega e apresentação: 01/06/2026

1. Objetivo

Desenvolver um sistema de gerenciamento para um hotel, com funcionalidades completas de **CRUD** (Create, Read, Update, Delete), aplicando os conceitos de modelagem de dados, comandos SQL, **views**, **stored procedures**, **triggers** e **transações**.

O projeto deve incluir uma **interface simples de consulta e operação (obrigatória)**, implementada em qualquer linguagem de programação, **exceto PHP**. Toda a comunicação com o banco de dados deve ser feita **exclusivamente via Stored Procedures e Views**, sem tráfego de comandos SQL "crus" pela rede.

Banco de Dados Obrigatório: Todo o desenvolvimento deve ser realizado utilizando **MySQL e o Workbench** ou superior.

2. Cenário Geral / Descrição do Sistema (Hotel)

O **Hotel do Mosquito** deseja informatizar sua gestão de reservas, hospedagem e faturamento. O hotel possui diversos **quartos**, cada um classificado por uma **categoria** (Standard, Superior, Luxo, Suíte e Master) com uma **capacidade** e uma **diária base**. Em períodos de alta temporada ou eventos na cidade, os preços das diárias são reajustados, e o sistema deve manter um **histórico completo de preços praticados**.

O sistema deve controlar as **reservas** realizadas por **clientes**, efetivadas por **funcionários** (recepcionistas). Quando o cliente chega, a reserva se transforma em uma **hospedagem** (check-in). Durante a estadia, o cliente pode consumir produtos e serviços do hotel (**frigobar, restaurante, lavanderia**), que serão lançados em sua conta. Ao final, é realizado o **check-out** com o cálculo do valor total.

Regra de Negócio Importante: O sistema deve garantir a segurança da informação. **Funcionários com perfil de "Recepcionista"** podem realizar reservas, check-ins, check-outs e consultar disponibilidade, **mas não podem visualizar relatórios de faturamento total ou lucro** (apenas gerentes podem).

Entidades Principais:

- **Categoria_Quarto:** (ID, Nome, Descricao)
- **Quarto:** (Número, Andar, Capacidade, Status, ID_Categoria)
- **Historico_Preco:** (ID, ID_Quarto, Preco_Praticado, Data_Inicio_Vigencia, Data_Fim_Vigencia)
- **Cliente:** (ID, Nome, CPF, Email, Telefone, Endereco)
- **Funcionario:** (ID, Nome, CPF, Cargo, Login, Senha_Hash, Perfil) Onde Perfil pode ser 'Recepcionista' ou 'Gerente'.
- **Reserva:** (ID, ID_Cliente, ID_Quarto, Data_Checkin_Prev, Data_Checkout_Prev, Status, ID_Funcionario)
- **Hospedagem:** (ID, ID_Reserva, Data_Checkin_Real, Data_Checkout_Real, Valor_Total_Diarias, Status)
- **Produto_Servico:** (ID, Nome, Preco_Venda, Tipo)
- **Consumo:** (ID, ID_Hospedagem, ID_Produto_Servico, Quantidade, Data_Hora)

3. Requisitos Obrigatórios (Entregáveis Técnicos)

3.1. Modelagem

- Criação do **MER (Modelo Entidade-Relacionamento)** completo.
- Geração do **modelo lógico relacional** (tabelas, chaves primárias, estrangeiras e cardinalidades).
- Entregar os diagramas (imagem ou PDF) junto com o projeto.

3.2. Banco de Dados (Script SQL)

- Criação de todas as tabelas com tipos de dados apropriados e restrições (NOT NULL, UNIQUE, FOREIGN KEY, CHECK).
- **Inserção de dados de exemplo (mínimo 15 registros nas tabelas principais):**
 - 5 Categorias de Quarto
 - 15 Quartos
 - 10 Clientes
 - 5 Funcionários (2 Gerentes, 3 Recepcionistas)
 - 10 Produtos/Serviços

- 10 Reservas
- 8 Hospedagens
- 15 Registros de Consumo
- Implementar regras de integridade referencial.
- O script deverá ser compatível e executável integralmente no MySQL.

3.3. CRUD via Stored Procedures (Proibido SQL cru na rede)

Fornecer scripts de **Stored Procedures** demonstrando as operações CREATE, READ, UPDATE e DELETE para **Quartos, Clientes, Funcionários e Reservas**. A interface do programa **DEVE** chamar essas procedures, não executar `INSERT` ou `UPDATE` diretamente.

3.4. Views (Mínimo 3)

Criar pelo menos três views úteis, como:

- 1.vw_quartos_disponiveis: Lista quartos com status 'Disponível' ou 'Limpo'.
- 2.vw_ocupacao_atual: Mostra as hospedagens em andamento (Check-in realizado, Check-out pendente).
- 3.vw_historico_reservas_cliente: Histórico de reservas e estadias por CPF.

3.5. Stored Procedures / Transações (Mínimo 2)

Criar pelo menos duas stored procedures utilizando transações (`BEGIN / COMMIT / ROLLBACK`), como:

- 1.sp_RealizarCheckIn(ID_Reserva, ID_Funcionario): Verifica se a reserva é válida, altera status do quarto para 'Ocupado', insere registro em `Hospedagem` e calcula valor estimado.
- 2.sp_RealizarCheckOut(ID_Hospedagem, ID_Funcionario): Calcula valor total das diárias + consumos, atualiza status do quarto para 'Manutenção/Limpeza', finaliza hospedagem. **Regra:** Se o funcionário for 'Recepcionista', não deve retornar o valor total (ou retornar mascarado), apenas confirma a operação. Gerentes podem ver o valor.

3.6. Interface de Consulta (Obrigatória)

Desenvolver interface em **Python (Tkinter/Flask)**, **Java (Swing/Spring)**, **C#** ou **JavaScript (Node.js/React)**. Não é permitido PHP.

Funcionalidades Mínimas da Interface:

- **Login** com verificação de perfil (Recepcionista vs. Gerente).
- Cadastrar, editar e excluir **Clientes**.
- Registrar uma **Reserva** utilizando a procedure criada.
- Realizar **Check-in** e **Check-out**.
- **Menu de Relatórios (Apenas Gerente):**
 - Taxa de Ocupação do mês.
 - **Top 10 Clientes Mais Assíduos** (maior número de diárias).
 - **Top 10 Quartos Mais Reservados**.
 - **Relatório de Faturamento Mensal**

3.7. Segurança: Login e Criptografia

- A tabela `Funcionario` deve armazenar a senha utilizando **Hash** (ex: SHA2 ou BCrypt via aplicação). Não armazenar senha em texto puro.
- A Stored Procedure de Login deve retornar os dados do funcionário (exceto senha) e seu perfil para a aplicação controlar o acesso às telas.

3.8. Triggers e Histórico de Preços

- Criação da tabela `Historico_Preco`.
- Criação de um **Trigger** `trg_atualiza_preco_quarto`:
 - **Evento:** AFTER UPDATE na tabela `Quarto` (coluna `Preco_Diaria`).
 - **Ação:** Insere um registro em `Historico_Preco` com o valor antigo, o novo valor e a data da alteração.
- Nota: A atualização de preços deve ser feita via procedure `sp_AtualizarPrecoQuarto` que executa o UPDATE e dispara a trigger automaticamente.

3.9. Apresentação do Trabalho

- **Apresentação em PPT ou PDF** contendo obrigatoriamente uma seção sobre "**Principais Dificuldades Encontradas pelo Grupo**" e como foram solucionadas.
- **Todos os integrantes devem apresentar** e demonstrar conhecimento sobre sua parte no código.

4. Entrega

Arquivos obrigatórios:

1. Script SQL completo (.sql)

- Criação de tabelas, índices, constraints, triggers, views e procedures.

- Inserção de dados de exemplo.
- Instruções para execução no MySQL.

2.Relatório Técnico (.pdf)

- Diagramas MER e Modelo Lógico.
- Explicação das procedures, views e triggers criados.
- Seção "Dificuldades Encontradas" (mínimo 1 página).
- Prints da interface (login, tela de reserva, relatório de ocupação).
- Casos de teste e resultados (ex: tentativa de login com senha errada, acesso de recepcionista a relatório de faturamento).

3.Código-fonte da Interface (arquivo compactado .zip)

- Código executável.
- Dependências e instruções de execução (ex: requirements.txt para Python, pom.xml para Java Maven).

4.Apresentação (PPT ou PDF)

- Slides para apresentação oral, contendo a análise das dificuldades do grupo.